Cache Controller with Snoopy Coherence for a Dual-Core System

Structure of Computer Systems · Project documentation

Student: Arthur Kantor

Co-Authored by: Claude Code

Group: 30232

Coordinating teacher: Mădălin Neagu

Academic year: 2026–2027

Technical University of Cluj-Napoca

Faculty of Automation and Computer Science, Computer Science Department

Table of Contents

- **1. Introduction**
  - 1.1 Project Proposal
- **2. Bibliographic Study**
  - 2.1 Cache Memory Organization
  - 2.2 Replacement Policies
  - 2.3 Write Policies
  - 2.4 The Cache Coherence Problem
  - 2.5 Snoopy Coherence and the MSI Protocol
  - 2.6 Shared Bus and Arbitration
  - 2.7 Modeling and Simulation in VHDL
- **3. Analysis**
  - 3.1 Requirements
  - 3.2 Cache Organization and Address Breakdown
  - 3.3 MSI Protocol Behaviour
  - 3.4 Coherence Scenarios
- **4. Design**
  - 4.1 System Architecture

    - 4.2 Interfaces
    - 4.3 Bus Protocol
    - 4.4 Processor-Side FSM
    - 4.5 Snoop FSM
    - 4.6 Replacement Unit
- **5. Implementation**
    - 5.1 Project Structure
    - 5.2 Package cache_pkg
    - 5.3 Cache Controller
    - 5.4 Replacement Unit
    - 5.5 Main Memory and Bus Arbiter
    - 5.6 Top Level and Bus Multiplexer
    - 5.7 Testbench and Simulation Log
    - 5.8 Simulation Viewer
- **6. Tests and Experiments**
    - 6.1 Test Setup
    - 6.2 Hit and Miss
    - 6.3 Replacement
    - 6.4 Write-Back
    - 6.5 Shared Read
    - 6.6 Invalidation
    - 6.7 Cache-to-Cache Transfer
    - 6.8 Simultaneous Writes
    - 6.9 Summary of Results
- **7. Conclusions**
    - 7.1 Results
    - 7.2 Elements Learned
    - 7.3 Further Improvements
- **8. Bibliography**

1. Introduction

1.1 Project Proposal

Project Description

The goal of this project is to design, describe in VHDL and verify by simulation a cache controller for a dual-core system. Each core has a private write-back cache. The two caches share one bus to main memory and are kept coherent by the MSI snoopy protocol.

Private caches keep frequently used data close to each core, but they also create several copies of the same memory line. Without a coherence mechanism, one core could keep reading an old value after the other core has written a new one. The project shows, at register-transfer level, how a cache controller serves its own core while it watches the bus to keep both copies consistent.

Specifications

Cache (one per core)

- All memory accesses of a core go through its cache.
- N-way set-associative organization. The number of sets, the associativity N and the line size are configurable.
- Replacement policy LRU or FIFO, selected with a generic.
- Write-back policy, with write-allocate on write misses.

Coherence and interconnect

- MSI protocol: every cache line is in one of the states Modified, Shared or Invalid.
- Each controller snoops the shared bus and responds to read (BusRd), read-exclusive (BusRdX) and invalidate (BusUpgr) transactions, and writes back a modified line when required.
- One shared bus to main memory, with an arbiter that grants it to one controller at a time (round-robin). A granted transaction completes before the next grant.

Verification

- The two cores are modeled in the testbench as request generators that issue read and write sequences.
- The simulation shows hits, misses, replacements and write-backs.
- The simulation shows three coherence scenarios:

1. Both cores read the same line: both copies end in state S.
2. One core writes a line cached by the other: the other copy is invalidated.
3. One core reads a line modified in the other cache: the owner supplies the line, which is also written back to memory (cache-to-cache transfer).

Objectives

- **Correctness:** every read returns the value of the most recent write to that address, made by either core.
- **Configurability:** the cache size, the associativity, the line size and the replacement policy can be changed without changing the logic.
- **Modularity:** the cache controller, the replacement unit, the arbiter and the memory are separate entities, so the design can later be extended (for example to MESI or to more cores).
- **Observability:** every scenario is checked automatically by the testbench (assertions), can be followed on the waveform, and can be replayed step by step in a separate simulation viewer.

2. Bibliographic Study

This chapter presents the concepts the design is based on: how a set-associative cache is organized, how it replaces and writes lines, and how a snoopy MSI protocol keeps two private caches coherent over a shared bus.

2.1 Cache Memory Organization

A cache keeps copies of main-memory blocks, called **lines**, close to the processor. It works because programs tend to reuse the same data (temporal locality) and data stored next to it (spatial locality) [1], [5].

In an **N-way set-associative** cache, the storage is divided into S sets of N lines each. A memory line can be placed in any of the N ways of exactly one set. Direct-mapped (N = 1) and fully associative (S = 1) caches are the two extremes [2].

For an A-bit address, a line size of L bytes and S sets, the cache capacity is C = S × N × L bytes, and the address is split into three fields:

- **Offset:** log2(L) bits, which select the byte inside the line.
- **Index:** log2(S) bits, which select the set.

- **Tag:** A − index − offset bits, which identify the line inside the set.

On an access, the index selects a set and the N tags stored in that set are compared in parallel. A match on a valid line is a **hit**; otherwise it is a **miss**. Each line stores its tag, its data, its coherence state and the information used by the replacement policy.

2.2 Replacement Policies

On a miss in a full set, one line must be evicted to make room. If the set still has an invalid way, that way is used first [1].

- **LRU (least recently used)** evicts the line that has not been used for the longest time. Exact LRU keeps an age of log2(N) bits for every way: the accessed way gets age 0 and every way younger than it ages by one. Its cost grows with N, so large caches use approximations (pseudo-LRU) [1].
- **FIFO (first in, first out)** evicts the line that was loaded first, using one pointer of log2(N) bits per set, advanced only when a line is filled. It is cheaper, but it ignores reuse: a frequently used line can still be evicted [5].

2.3 Write Policies

On a write hit, a **write-through** cache updates memory on every write, while a **write-back** cache updates only the cache and marks the line as modified (dirty). Memory is updated only when the line is evicted [1]. Write-back sends much less traffic to memory, which matters when two cores share one bus [6].

On a write miss, **write-allocate** first brings the line into the cache and then writes it, while **no-write-allocate** writes directly to memory. Write-back is normally paired with write-allocate, so later writes to the same line hit [2]. In MSI, the Modified state plays the role of the dirty bit.

2.4 The Cache Coherence Problem

With private write-back caches, two copies of the same line can become different. If both cores cache address X and core 0 writes it, core 1 keeps reading the old value from its own cache [3]. A coherence protocol prevents this by enforcing two rules [3]:

- **Single writer, multiple readers:** at any moment, a line is either writable in exactly one cache, or read-only in any number of caches.
- **Data value:** a read returns the value of the most recent write to that line.

Coherence protocols are either **snooping** or **directory-based**. In snooping, every request is broadcast on a shared medium that all controllers observe; this is simple and fast for a few cores, but it does not scale. Directory protocols record which caches hold each line and scale to many cores [1], [3]. For two cores on one bus, snooping is the natural choice.

2.5 Snoopy Coherence and the MSI Protocol

Goodman's write-once protocol was the first to keep caches coherent by snooping the bus [6]. Later protocols are named after their states: MSI, MESI (the Illinois protocol [7]) and MOESI. Archibald and Baer compared them by simulating a multiprocessor [8]. **MSI** is the basic invalidation-based protocol and the one used in this project, in the notation of Culler et al. [4].

State	Meaning	The other cache may hold	Memory up to date
M (Modified)	Only valid copy, written locally	Nothing (I)	No
S (Shared)	Clean, read-only copy	S or I	Yes
I (Invalid)	Line absent or invalidated	M, S or I	—

Bus transaction	Issued on	Response of the other cache
BusRd	Read miss	A copy in M is flushed and becomes S
BusRdX	Write miss (write-allocate)	Any copy is invalidated; a copy in M is flushed first
BusUpgr	Write hit on a line in S	The copy in S is invalidated; no data is transferred
Flush	The owner supplies or evicts a line in M	Memory is updated

When the owner flushes a line in response to BusRd or BusRdX, both memory and the requesting cache can take the data from the bus (**cache-to-cache transfer**), which saves a second memory access [4]. A known cost of MSI is that reading and then writing a line

that no other cache holds needs two transactions (BusRd, then BusUpgr). The Exclusive state of MESI removes the second one [7]; it is outside the scope of this project.

2.6 Shared Bus and Arbitration

The bus is a single shared resource: each controller raises a request line and the arbiter asserts exactly one grant [2]. A fixed-priority arbiter is the simplest, but it can starve the low-priority core. A **round-robin** arbiter alternates between requesters and is fair.

The bus also orders the coherence events: the order in which requests are granted is the order in which all caches see the writes to a line [4]. With an **atomic bus**, where a transaction completes before the next grant, the main remaining race is two caches with the same line in S that both try to write it at the same time. The controller that loses arbitration sees the winner's invalidation and must reissue its request as BusRdX [4].

2.7 Modeling and Simulation in VHDL

The processor-side and snoop-side controllers map naturally onto finite-state machines, and the tag, data and state storage onto arrays of records indexed by set and way [9]. Constants and generics set the number of sets, the associativity, the line size and the replacement policy [9], [10]. Testbench processes model the cores, and assertions check the expected values and line states, so every scenario verifies itself in simulation [9].

3. Analysis

This chapter turns the specification into concrete requirements, fixes the cache configuration used in simulation, and describes exactly what the MSI protocol must do in each situation.

3.1 Requirements

ID	Requirement	Checked in
R1	A read of a line that is in the cache (hit) is served without using the bus	6.2
R2	A read or write miss brings the whole line into the cache (write-allocate)	6.2, 6.4
R3	When a set is full, the victim is chosen by the selected policy (LRU or FIFO)	6.3

ID	Requirement	Checked in
R4	A modified line is written back to memory before it is replaced	6.4
R5	Two caches can hold the same line at the same time for reading (state S)	6.5
R6	A write to a shared line invalidates the other copy	6.6
R7	A read of a line modified in the other cache returns the new value	6.7
R8	When both cores write the same line at the same time, no write is lost	6.8

3.2 Cache Organization and Address Breakdown

The simulation uses a small configuration, so that replacements appear after only a few accesses and the waveforms stay readable. All sizes are powers of two and are set in one place (see 5.2).

Parameter	Value
Address width	32 bits (byte address)
Word size	32 bits (4 bytes)
Line size L	16 bytes = 4 words
Number of sets S	4
Associativity N	2 ways
Capacity of one cache	4 × 2 × 16 = 128 bytes
Main memory	256 lines × 16 bytes = 4 KiB
Memory latency	4 clock cycles

With these values, the 32-bit address is split as follows:

Bits	31 … 6	5 … 4	3 … 2	1 … 0
Field	Tag (26 bits)	Index (2 bits)	Word in line (2 bits)	Byte in word (2 bits)

For example, address **0x034** = 0b11 0100 has tag 0, index 3 and word 1, so it is the second word of the line stored in set 3. Addresses that differ by a multiple of 64 bytes (0x000, 0x040, 0x080 …) map to the same set, which is how the tests force replacements.

3.3 MSI Protocol Behaviour

Every line changes state because of two kinds of events: requests from its own core (PrRd, PrWr) and transactions issued by the other cache and seen on the bus.

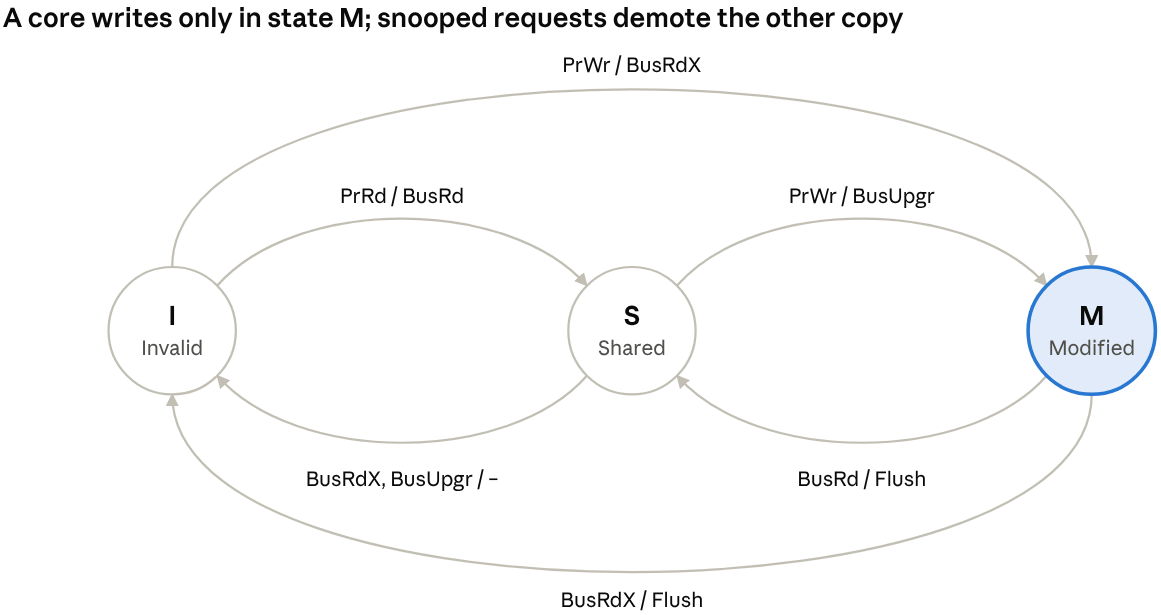


Figure 2. MSI state diagram

Arcs above the states are requests from the own core; arcs below are transactions of the other cache, seen on the bus. Each label reads *event / bus action*. Hits keep the state, and evicting a line goes to I (with a Flush first if the line is in M).

Requests from the own core

Current state	Read (PrRd)	Write (PrWr)	Line chosen as victim
I	Miss: issue BusRd, go to S	Miss: issue BusRdX, go to M	—
S	Hit, stay in S	Issue BusUpgr, go to M	Drop the line, go to I
M	Hit, stay in M	Hit, stay in M	Flush (write back), go to I

Transactions issued by the other cache

Current state	BusRd	BusRdX	BusUpgr
I	Nothing	Nothing	Nothing
S	Nothing	Go to I	Go to I
M	Flush the line, go to S	Flush the line, go to I	Cannot happen (the other cache holds the line in S, so this one cannot hold it in M)

3.4 Coherence Scenarios

The three coherence scenarios required by the assignment follow directly from the tables above. Line X starts in state I in both caches.

Scenario	Sequence	Bus transactions	Final state (cache 0, cache 1)
Shared read	Core 0 reads X, then core 1 reads X	BusRd (from memory), BusRd (from memory)	S, S
Invalidation	After the shared read, core 0 writes X	BusUpgr	M, I
Cache-to-cache transfer	After the invalidation, core 1 reads X	BusRd; cache 0 flushes X, cache 1 and memory take it	S, S

In the last scenario, memory does not hold the newest value of X before the transfer. If cache 0 did not answer the BusRd, core 1 would read the old value from memory; this is exactly the error the protocol prevents.

4. Design

4.1 System Architecture

The system has one top-level entity, `dual_core_system`, which contains two identical cache controllers, a bus arbiter and a main memory. The two cores are not part of the design: the testbench plays their role.

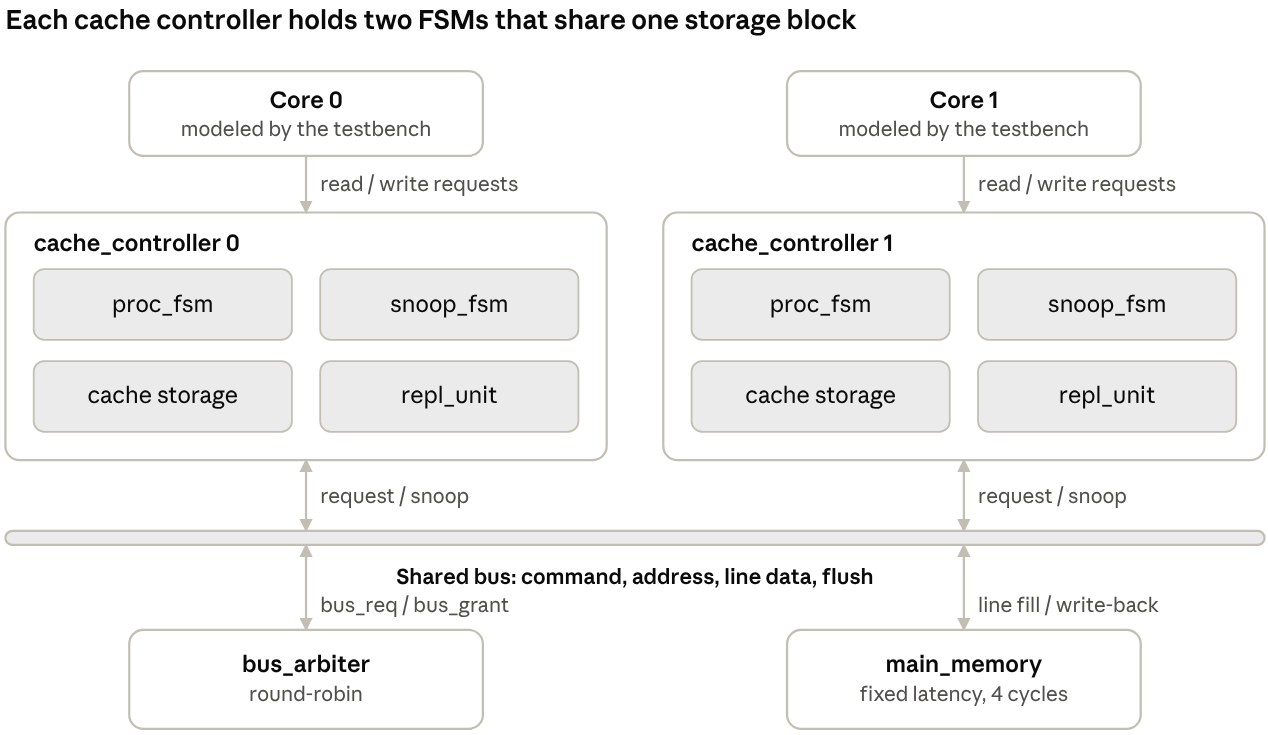


Figure 3. System architecture

Each core talks only to its own cache controller. Everything else (misses, upgrades and write-backs) goes through the shared bus, which the arbiter gives to one controller at a time.

Component	Instances	Role
`dual_core_system`	1	Top level: connects the controllers, the arbiter and the memory, and builds the shared bus
`cache_controller`	2	One private cache: storage, processor-side FSM, snoop FSM and replacement unit
processor-side FSM (`proc_fsm`)	1 per controller	Serves the own core: lookup, hit or miss, write-back, bus request, line fill
snoop FSM (`snoop_fsm`)	1 per controller	Watches the bus and applies the MSI transitions to the local copy
cache storage	1 per controller	Tag, data and MSI state of every line, indexed by set and way
`repl_unit`	1 per controller	LRU ages or FIFO pointers; chooses the victim way
`bus_arbiter`	1	Round-robin grant between the two controllers
`main_memory`	1	Memory model with a fixed latency; serves line fills and write-backs

4.2 Interfaces

Configuration (the same for both caches)

Name	Value used	Meaning
`OFFSET_W`	4	Line size = 2^4 = 16 bytes
`INDEX_W`	2	Number of sets = 2^2 = 4
`WAY_W`	1	Associativity = 2^1 = 2 ways
`MEM_LATENCY`	4	Memory access time in clock cycles
`POLICY` (generic)	LRU	Replacement policy: LRU or FIFO

Ports of `cache_controller`

Group	Signal	Direction	Width	Meaning
Core	`cpu_req`	in	1	The core issues a request
Core	`cpu_we`	in	1	1 = write, 0 = read
Core	`cpu_addr`	in	32	Byte address
Core	`cpu_wdata`	in	32	Data to write
Core	`cpu_rdata`	out	32	Data read
Core	`cpu_ready`	out	1	The request is complete (one-cycle pulse)
Bus, to arbiter	`bus_req`	out	1	The controller needs the bus
Bus, to arbiter	`bus_grant`	in	1	The bus is given to this controller
Bus, driven	`cmd_o`	out	enum	Command: Idle, BusRd, BusRdX, BusUpgr, Flush
Bus, driven	`addr_o`	out	32	Line address of the command
Bus, driven	`data_o`	out	128	A whole line (write-back or snoop flush)
Bus, driven	`flush_o`	out	1	This controller supplies a modified line requested by the other one
Bus, observed	`bus_cmd`, `bus_addr`, `bus_data`	in	enum, 32, 128	What is currently on the shared bus
Bus, observed	`bus_done`	in	1	The current transaction is finished
Debug	`dbg_hit`, `dbg_miss`	out	1	One-cycle pulse on every hit or miss

Group	Signal	Direction	Width	Meaning
Debug	`probe_addr`, `probe_state`	in, out	32, enum	MSI state of the line at any address (used by the testbench checks)
Debug	`dbg_cache`	out	whole cache	Contents of the cache (used by the simulation log)

Each controller drives its own bus outputs (`cmd_o`, `addr_o`, `data_o`, `flush_o`). The top level selects which of them reaches the shared bus, so no signal has two drivers.

4.3 Bus Protocol

The bus is **atomic**: once a controller is granted the bus, its transaction finishes before the arbiter grants the bus again. One transaction runs in five steps:

1. The requesting controller asserts `bus_req`.
2. The arbiter asserts `bus_grant` for one controller (round-robin when both request).
3. The granted controller drives the command and the line address (and the line data for Flush).
4. The other controller's snoop FSM looks up the address in its own cache:
   - line in M and BusRd or BusRdX: it asserts `flush_o` and drives the line on the bus;
   - line in S and BusRdX or BusUpgr: it invalidates its copy;
   - otherwise it does nothing.
5. After `MEM_LATENCY` cycles, memory asserts `bus_done` for one cycle:
   - if a cache flushed the line, memory stores it and the requester takes the same data from the bus (cache-to-cache transfer);
   - otherwise memory supplies the line (BusRd, BusRdX) or stores it (Flush);
   - BusUpgr carries no data.

With `MEM_LATENCY = 4`, every transaction holds the bus for **6 clock cycles**, from grant to release. BusUpgr uses the same timing: the extra cycles give the other controller time to invalidate its copy before the writer continues.

The eviction of a line in M is its own Flush transaction, done before the request for the new line.

Upgrade race. If both caches hold a line in S and both write it at the same time, both request BusUpgr. The winner invalidates the loser's copy, so the loser's processor-side FSM changes its pending command from BusUpgr to BusRdX before it gets the bus. This case is tested in 6.8.

4.4 Processor-Side FSM

The processor-side FSM serves one request of its core at a time. After a miss it fills the line and returns to COMPARE, where the access now hits, so every read and write is completed in the same state.

Every miss ends with a line fill and a second lookup that hits

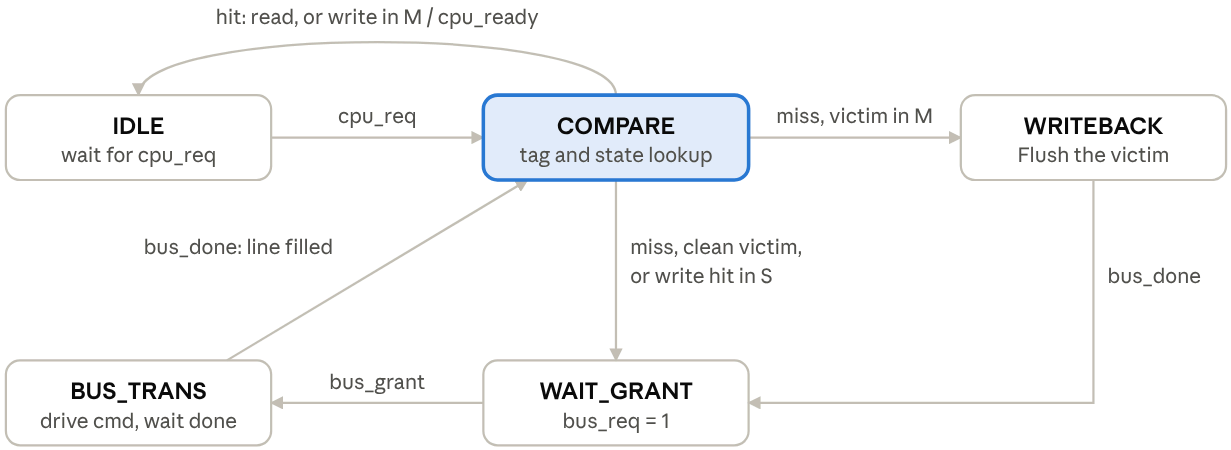


Figure 4. Processor-side FSM

State	Actions	Next state
IDLE	Latch `cpu_addr`, `cpu_we`, `cpu_wdata`	COMPARE when `cpu_req` = 1
COMPARE	Split the address into tag, index and offset; compare the N tags of the set	**Read hit** (S or M) or **write hit in M**: return or write the word, update the replacement unit, `cpu_ready` = 1, go to IDLE. **Write hit in S**: command BusUpgr, go to WAIT_GRANT. **Miss**: choose the victim way; victim in M: go to WRITEBACK; otherwise command BusRd (read) or BusRdX (write), go to WAIT_GRANT

State	Actions	Next state
WRITEBACK	Request the bus and issue Flush with the victim line; the victim becomes I	WAIT_GRANT on `bus_done`
WAIT_GRANT	`bus_req` = 1; if the line was invalidated while waiting, BusUpgr becomes BusRdX	BUS_TRANS on `bus_grant`
BUS_TRANS	Drive the command; on `bus_done` store the line and the tag, and set the state to S (BusRd) or M (BusRdX, BusUpgr)	COMPARE on `bus_done`

While the snoop FSM is busy, COMPARE waits. This way the two FSMs never change the same line in the same clock cycle.

4.5 Snoop FSM

The snoop FSM reacts only to transactions issued by the other controller. It ignores the commands of its own controller and every Flush (an eviction only concerns memory).

Only a line held in M makes the snooping cache drive the bus

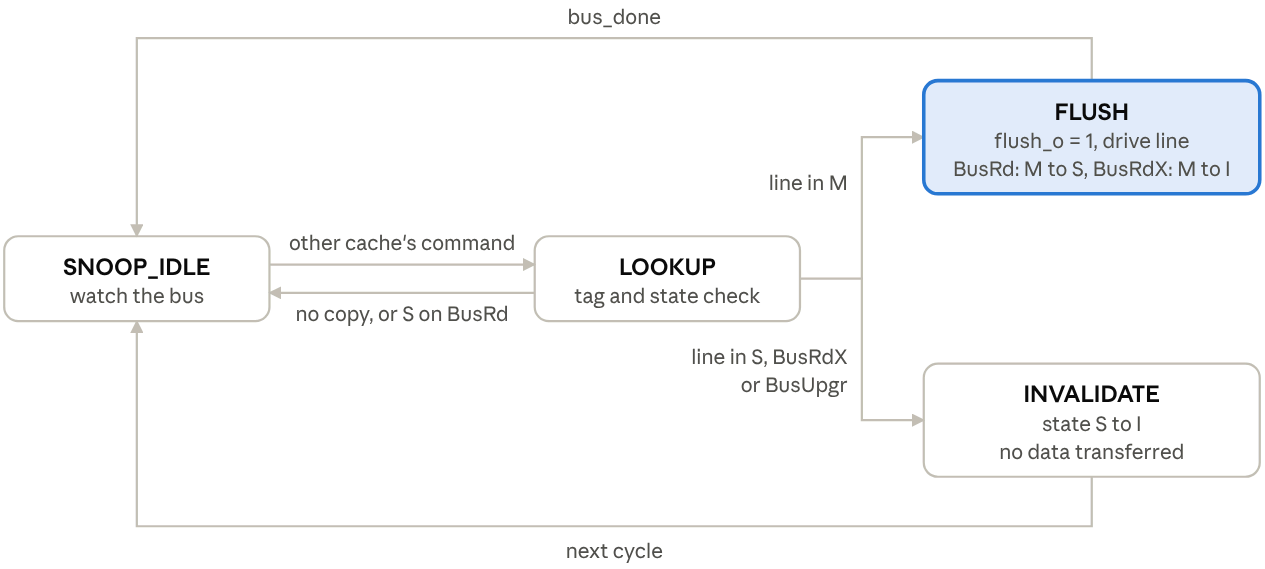


Figure 5. Snoop FSM

State	Actions	Next state
SNOOP_IDLE	Wait for a BusRd, BusRdX or BusUpgr from the other controller; latch its address and command	LOOKUP
LOOKUP	Look up the address in the own cache	FLUSH if the line is in M and the command is BusRd or BusRdX; INVALIDATE if the line is in S and the command is BusRdX or BusUpgr; otherwise SNOOP_IDLE
FLUSH	`flush_o` = 1, drive the line on the bus; on `bus_done` the line becomes S (BusRd) or I (BusRdX)	SNOOP_IDLE on `bus_done`
INVALIDATE	The line becomes I	SNOOP_IDLE

4.6 Replacement Unit

On a miss, the controller first looks for a way in state I in the selected set. Only if all ways are valid does it ask the replacement unit for a victim.

- **LRU:** every way of every set has an age between 0 and N − 1. On every hit or fill, the accessed way gets age 0 and the ways younger than it grow older by one. The victim is the way with age N − 1.
- **FIFO:** every set has one pointer to the next way to replace. The pointer moves forward only when that way is filled, so hits do not change the order.

Example with 2 ways: core 0 reads A, then B, then A again, and then C, which maps to the same set. With LRU, B is evicted, because A was used more recently. With FIFO, A is evicted, because it was loaded first. Scenario 6.3 checks exactly this case.

5. Implementation

The hardware is written in VHDL and simulated in Xilinx Vivado (XSim). The code is compatible with both VHDL-93 and VHDL-2008, and it was also checked with the GHDL simulator. The simulation viewer is a separate C++17 program.

5.1 Project Structure

File	Lines	Content
`cache_pkg.vhd`	104	Configuration constants, types and address helper functions
`main_memory.vhd`	91	Main memory model
`repl_unit.vhd`	93	LRU and FIFO replacement
`cache_controller.vhd`	302	Cache storage, processor-side FSM, snoop FSM
`bus_arbiter.vhd`	43	Round-robin arbiter
`dual_core_system.vhd`	160	Top level and bus multiplexer
`tb_dual_core.vhd`	523	Testbench: the two cores, the checks and the simulation log
`viewer/src/*`	913	Simulation viewer (C++, Dear ImGui)

The hierarchy is: `tb_dual_core` → `dual_core_system` → two `cache_controller` instances (each with one `repl_unit`), one `bus_arbiter` and one `main_memory`. `cache_pkg` is not a component; every file includes it with `use work.cache_pkg.all`.

5.2 Package cache_pkg

The package holds everything the other files share. The cache sizes are given as numbers of bits, so they are always powers of two, and all other sizes are derived from them:

```
constant OFFSET_W  : positive := 4;
constant INDEX_W   : positive := 2;
constant WAY_W     : positive := 1;
constant LINE_SIZE : positive := 2**OFFSET_W;
constant NUM_SETS  : positive := 2**INDEX_W;
constant ASSOC     : positive := 2**WAY_W;
constant TAG_W     : positive := ADDR_W - INDEX_W - OFFSET_W;
```

A cache line is a record, a set is an array of lines, and the whole cache is an array of sets:

```
type msi_t is (I, S, M);

type cache_line_t is record
  state : msi_t;
  tag   : tag_t;
  data  : line_data_t;
end record;

type cache_set_t   is array (0 to ASSOC-1) of cache_line_t;
type cache_array_t is array (0 to NUM_SETS-1) of cache_set_t;
```

The MSI state also replaces the usual valid and dirty bits: I means invalid, M means modified. The package also defines the bus commands (`B_IDLE`, `B_RD`, `B_RDX`, `B_UPGR`, `B_FLUSH`) and small functions that split an address (`get_tag`, `get_index`, `get_word`) and search a set (`find_way` returns the way that holds a tag, or −1 on a miss; `find_free` returns the first way in state I).

5.3 Cache Controller

Storage. The cache is one signal of type `cache_array_t`, so cache 0 and cache 1 each have their own copy. A line is accessed as `cache(index)(way)`.

Lookup. The tag comparison is combinational. It is done for the processor-side request and, in parallel, for the address latched by the snoop FSM:

```
req_idx  <= get_index(req_addr);
req_tag  <= get_tag(req_addr);
p_way    <= find_way(cache(req_idx), req_tag);
free_way <= find_free(cache(req_idx));
sn_idx   <= get_index(sn_addr);
s_way    <= find_way(cache(sn_idx), get_tag(sn_addr));
```

One process for both FSMs. In VHDL, a signal cannot be written by two processes. Both FSMs must change the cache, so they are written as two separate state machines (`pst` and `sst`) inside the same clocked process, one after the other. The processor-side part writes a line only in COMPARE (when the snoop FSM is idle) or during its own bus transaction, when the snoop FSM cannot be active, so the two parts never write the same line in the same cycle.

The snoop decision in LOOKUP follows the table of section 3.3 directly:

```
when LOOKUP =>
  if s_way >= 0 and cache(sn_idx)(s_way).state = M and
     (sn_cmd = B_RD or sn_cmd = B_RDX) then
    sn_way <= s_way;
    sst    <= FLUSH;
  elsif s_way >= 0 and cache(sn_idx)(s_way).state = S and
        (sn_cmd = B_RDX or sn_cmd = B_UPGR) then
    sn_way <= s_way;
    sst    <= INVALIDATE;
  else
    sst <= SNOOP_IDLE;
  end if;
```

Outputs. `bus_req` is 1 in WRITEBACK, WAIT_GRANT and BUS_TRANS. `cmd_o` is Flush in WRITEBACK and the pending command in WAIT_GRANT and BUS_TRANS. `data_o` carries the line being flushed by the snoop FSM, or else the victim line being written back.

5.4 Replacement Unit

`repl_unit` receives the set index, the accessed way and two pulses, `touch` (hit) and `fill` (line loaded). It returns the victim way of the current set. The policy is chosen with the generic `POLICY` and a `generate` statement, so only one of the two versions is built. The LRU update:

```
old := ages(set_idx)(way);
for wi in 0 to ASSOC-1 loop
  if wi = way then
    ages(set_idx)(wi) <= 0;
  elsif ages(set_idx)(wi) < old then
    ages(set_idx)(wi) <= ages(set_idx)(wi) + 1;
  end if;
end loop;
```

The victim is the way whose age is `ASSOC-1`. The FIFO version keeps one pointer per set and moves it forward on `fill` when the filled way is the one it points to.

5.5 Main Memory and Bus Arbiter

Main memory is an array of 256 lines. At start-up, each 32-bit word holds its own address (the word at 0x044 contains 0x00000044), so any read can be checked easily. When a command appears on the bus, memory waits `MEM_LATENCY` cycles and then:

- stores the line from the bus, for Flush or when a cache flushed the line;
- otherwise returns the line, for BusRd and BusRdX;

and asserts `bus_done` for one cycle. A check at start-up stops the simulation if `MEM_LATENCY` is below 4, because the snoop FSM needs 3 cycles to answer.

The bus arbiter keeps the grant until `bus_done`. When both controllers request at the same time, it gives the bus to the one that did not have it last.

5.6 Top Level and Bus Multiplexer

`dual_core_system` instantiates the components and builds the shared bus with multiplexers. The command and the address come from the granted controller. The data comes from the controller that writes back or flushes a line, and otherwise from memory:

```
bus_cmd <= cmd0 when grant(0) = '1' else
           cmd1 when grant(1) = '1' else
           B_IDLE;

bus_data <= data0 when (grant(0) = '1' and cmd0 = B_FLUSH) or flush0 = '1'
else
            data1 when (grant(1) = '1' and cmd1 = B_FLUSH) or flush1 = '1'
else
            mem_rdata;
```

This replaces a tri-state bus: in simulation, signals with several drivers and the value 'Z' are a common source of errors.

5.7 Testbench and Simulation Log

The testbench `tb_dual_core` generates a 100 MHz clock, holds reset for 3 cycles and runs the 7 scenarios of chapter 6 from one process. Each core is driven through the procedure `cpu_op`, which sets the request and waits until `cpu_ready` = 1. On top of it, three procedures make the checks:

- `rd(core, addr, expected)` reads a word and reports an error if the value differs;
- `wr(core, addr, value)` writes a word;
- `chk_state(core, addr, expected)` reads the MSI state of a line through `probe_addr` and `probe_state`.

At the end, the testbench prints `ALL TESTS PASSED` or the number of failed checks, and stops the clock, so **Run All** ends the simulation by itself.

A second process writes the file `cache_log.jsonl` with `std.textio`. Every line is one event in JSON format: a scenario start, a completed core request, the start and the end of a bus transaction, or a change in any cache line (with the full contents of both caches). For example, the end of the cache-to-cache transfer of scenario 6:

```
{"t":1305,"type":"bus_end","core":1,"cmd":"BusRd","addr":"00000020","source":
"cache0","mem_write":true,"data":
["BBBB0002","00000024","00000028","0000002C"]}
```

5.8 Simulation Viewer

The waveform shows every signal, but it is hard to follow which line changed state and why. The viewer reads `cache_log.jsonl` and replays the simulation step by step.

Part	Technology
Language	C++17
Graphical interface	Dear ImGui 1.91.5
Window and rendering	GLFW 3.4, OpenGL 3
Log parsing	nlohmann/json 3.11.3
Build	CMake 3.20 or newer (downloads the libraries automatically)

Module	Role
`main.cpp`	Creates the window and runs the main loop
`log_model`	Reads the log and builds the list of steps with the cache contents at each step
`describe`	Writes the explanation of each step and the list of changed lines
`player`	Navigation: next, previous, play, speed
`diagram`	Draws the cores, caches, bus and memory, and highlights the active data path

Module	Role
`tables`	Draws the two caches and the written-back memory lines
`app`	Puts the window together: toolbar, scenario and step list, step view

For every step, the viewer shows the data path, a sentence explaining the step (for example *"BusRd done: cache 0 held line 0x020 in M, so it sends the data directly to cache 1"*), the lines that changed state, and both caches as tables with the states colored (M red, S green, I gray).

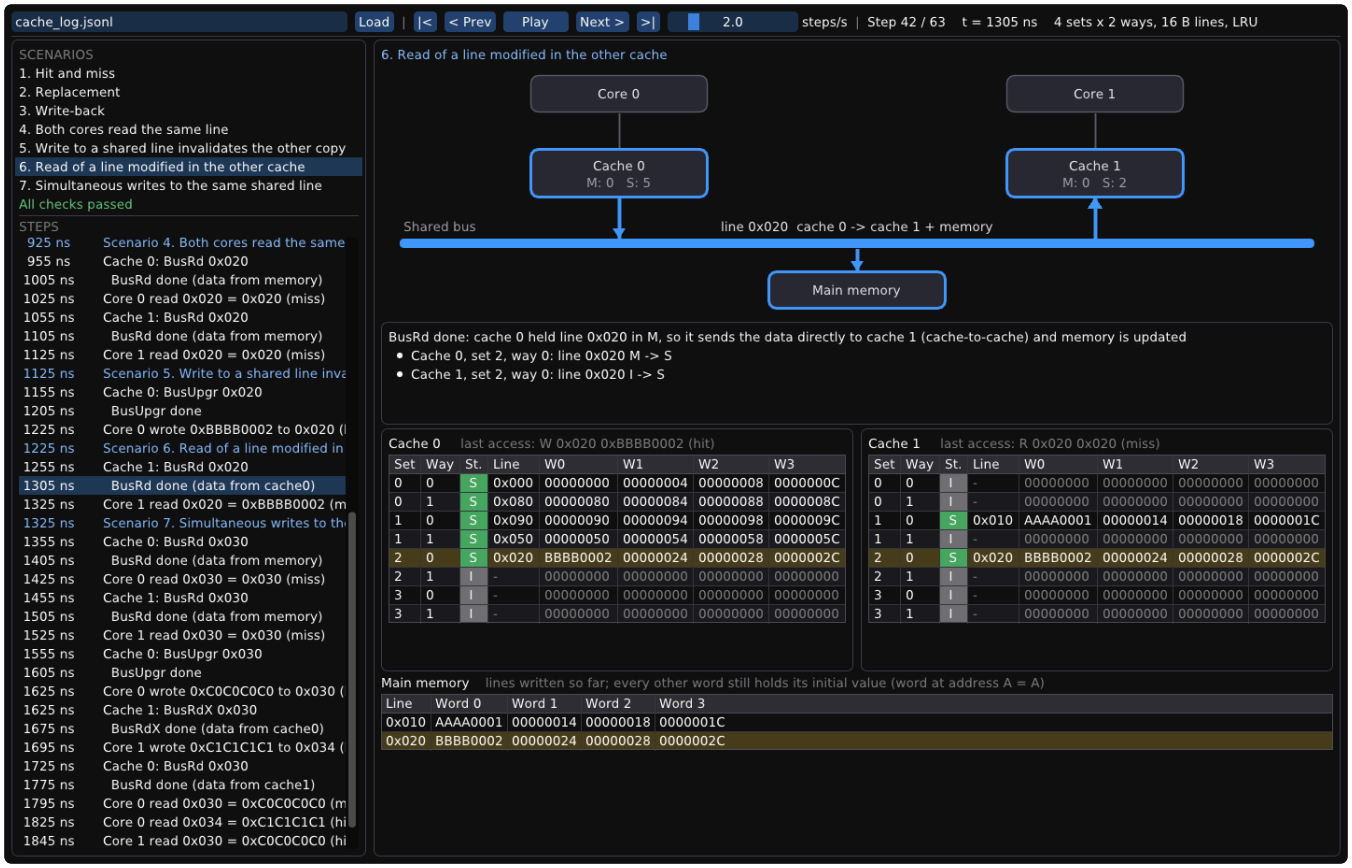


Figure 6. The simulation viewer at the end of the cache-to-cache transfer of scenario 6

The viewer is started with the path of the log: `./build/cache_viewer <path to cache_log.jsonl>`. The arrow keys move between steps and Space starts or stops the replay.

6. Tests and Experiments

6.1 Test Setup

All tests run in one simulation of `tb_dual_core`, with the configuration from section 3.2 (4 sets, 2 ways, 16-byte lines, LRU, memory latency 4 cycles) and a 10 ns clock. The 7 scenarios run one after the other, so each one starts from the cache contents left by the previous ones. They use different sets where possible, so they do not disturb each other.

Every scenario is **self-checking**: each read is compared with the expected value, and the MSI state of the important lines is checked in both caches. Because memory starts with each word equal to its own address, a read of 0x044 must return 0x00000044 unless some core has written that word.

The whole run takes about 1.87 µs of simulated time. Vivado stops a simulation at 1000 ns by default, so it must be started with **Run All** (or with `xsim.simulate.runtime` set to `all`). The end of the Tcl console output is:

```
Note: 1. hit and miss
Note: 2. replacement
Note: 3. write-back
Note: 4. both cores read the same line
Note: 5. write to a shared line invalidates the other copy
Note: 6. read of a line modified in the other cache
Note: 7. simultaneous writes to the same shared line
Note: ALL TESTS PASSED
```

In the waveforms below, the states of the two FSMs are shortened: CMP = COMPARE, WB = WRITEBACK, WAIT = WAIT_GRANT, BUS = BUS_TRANS, LOOK = LOOKUP, INV = INVALIDATE. The last rows show the MSI state of the lines involved (M red, S green, I white).

6.2 Hit and Miss

Sequence: core 0 reads 0x000, then 0x004 (the same line).

Expected: the first read misses and brings the line from memory; the second read hits without using the bus.

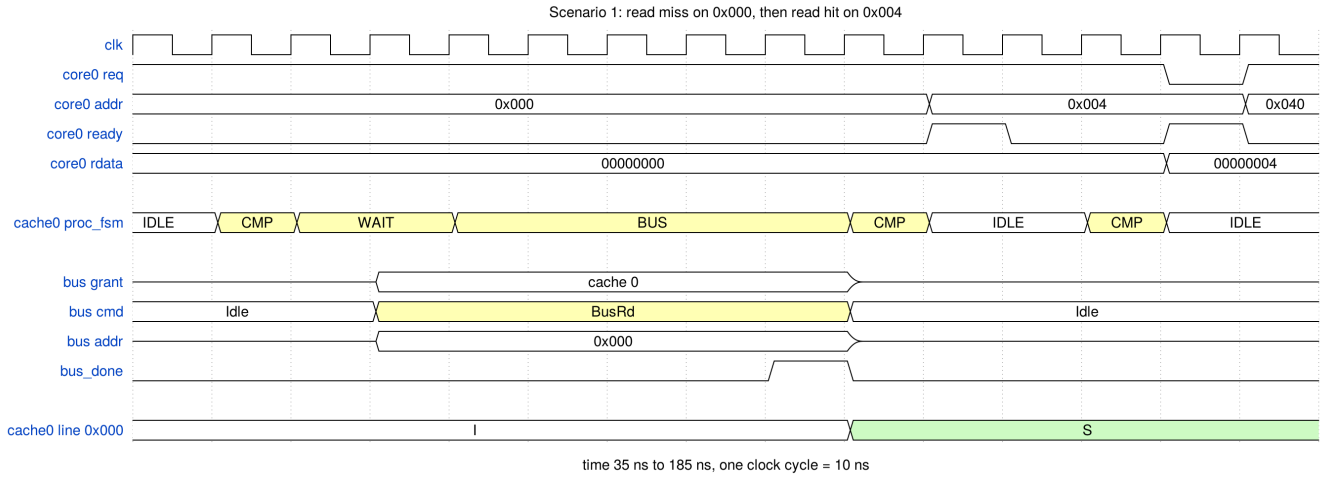


Figure 7. Read miss on 0x000, then read hit on 0x004

The first read finds no valid tag in set 0 (COMPARE), requests the bus (WAIT_GRANT), and receives the line from memory after a BusRd that holds the bus for 6 cycles. The line becomes S, the FSM goes back to COMPARE, where the access now hits, and `cpu_ready` returns 0x00000000. The second read hits in COMPARE and returns 0x00000004 one cycle later; the bus stays idle.

Result: values correct, 1 miss and 1 hit counted. **Passed.**

6.3 Replacement

Sequence: core 0 reads 0x040, then 0x000 again, then 0x080. Together with 0x000 from 6.2, all three lines map to set 0, which has only 2 ways.

Expected: 0x040 fills the second way. Reading 0x000 makes 0x040 the least recently used line. Reading 0x080 finds the set full, so LRU must evict 0x040.

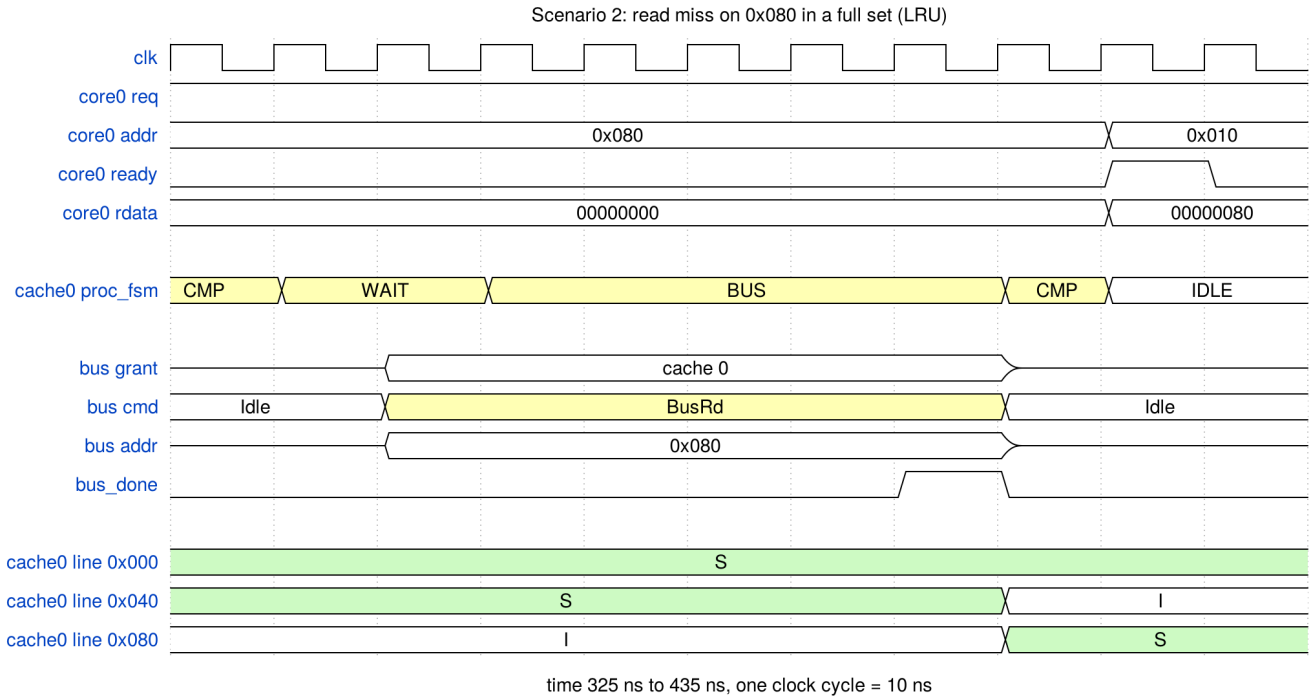


Figure 8. Read miss on 0x080 in a full set: LRU evicts 0x040

On the miss for 0x080, no way of set 0 is in state I, so the replacement unit chooses the victim: way 1, which holds 0x040. The victim is in S (clean), so it is simply overwritten, without a write-back. At the end, 0x000 and 0x080 are in S and 0x040 is I.

Result: with `POLICY = LRU` the evicted line is 0x040; the same test run with `POLICY = FIFO` evicts 0x000 (the first line loaded), as expected. **Passed** for both policies.

6.4 Write-Back

Sequence: core 0 writes 0xAAAA0001 to 0x010 (a miss), reads 0x050, then reads 0x090. All three lines map to set 1. Then core 1 reads 0x010.

Expected: the write brings the line with BusRdX and leaves it in M. When 0x090 needs a way, the victim is 0x010, which is modified, so it must be written back to memory before it is replaced. Core 1 must then read the new value from memory.

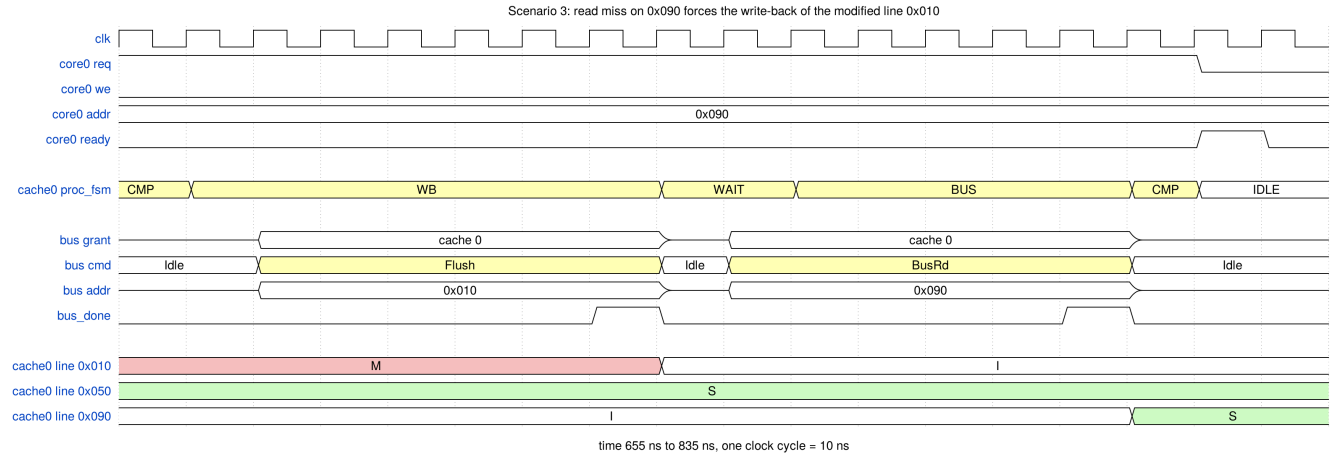


Figure 9. The read miss on 0x090 first writes back the modified line 0x010

The miss on 0x090 goes from COMPARE to WRITEBACK, because the victim (0x010) is in M. Cache 0 first issues a Flush, which writes the line to memory, and the line becomes I. Only then does it request the bus again and issue the BusRd for 0x090. This miss takes 16 cycles instead of 9, because it needs two bus transactions.

Result: 0x010 is I in cache 0 after the eviction, and core 1 reads 0xAAAA0001 from memory, which proves the write-back. **Passed.**

6.5 Shared Read

Sequence: core 0 reads 0x020, then core 1 reads 0x020.

Expected: both caches end with the line in S, and both reads return 0x00000020.

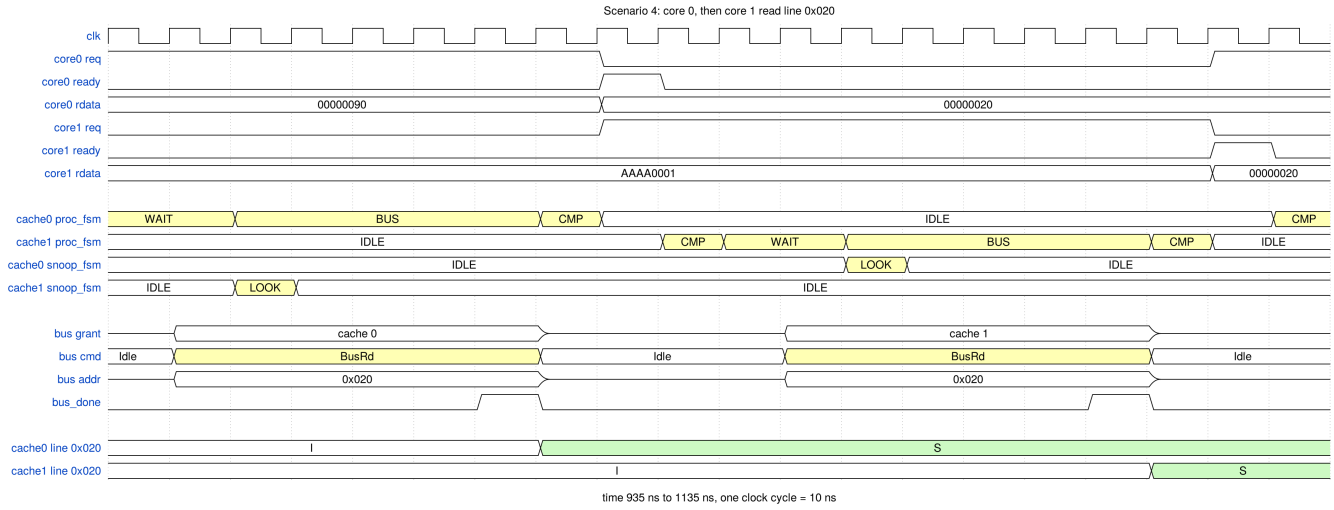


Figure 10. Core 0, then core 1 read line 0x020

Both reads miss and issue a BusRd. During the second BusRd, the snoop FSM of cache 0 goes to LOOKUP, finds the line in S and does nothing (a read does not affect a clean copy). Memory supplies the line to cache 1.

Result: both reads return 0x00000020, and the line is in S in both caches. **Passed.**

6.6 Invalidation

Sequence: core 0 writes 0xBBBB0002 to 0x020, while both caches hold the line in S.

Expected: the write hits a line in S, so cache 0 needs exclusive ownership: it issues BusUpgr, cache 1 invalidates its copy, and cache 0 moves to M.

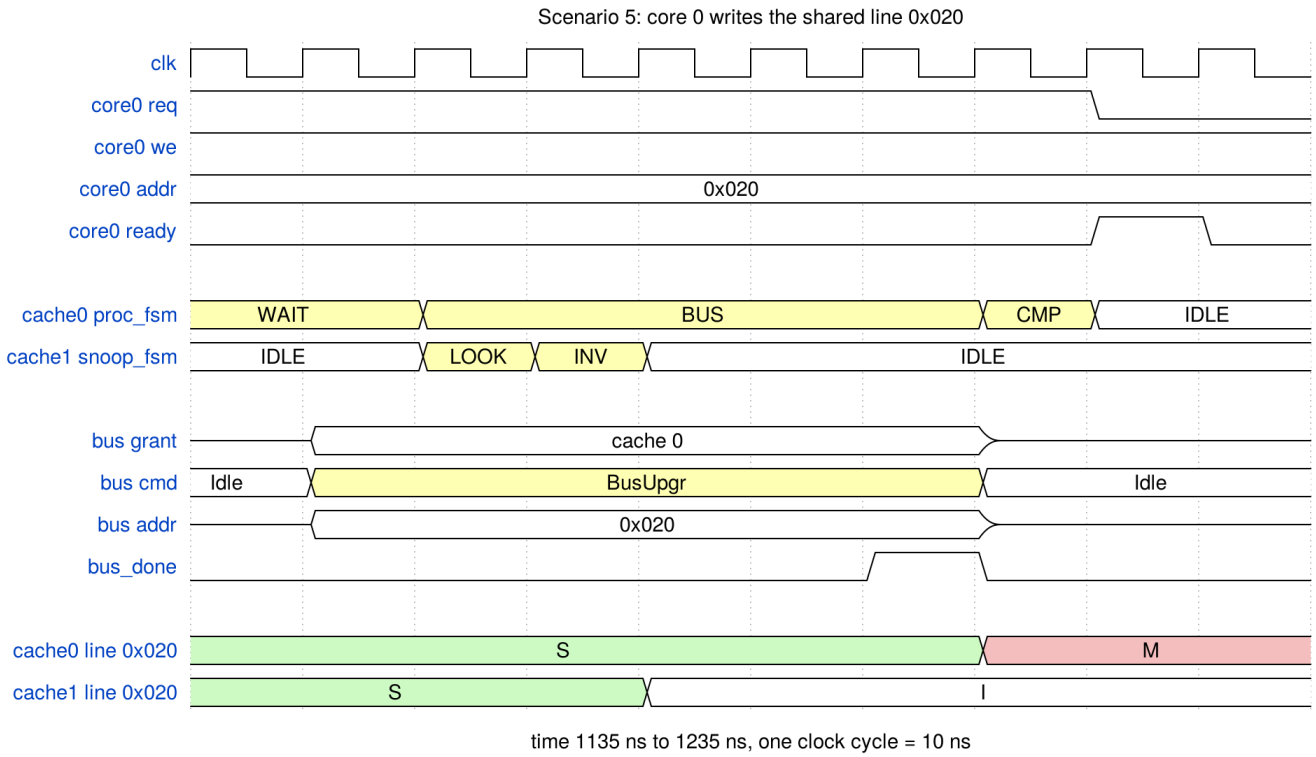


Figure 11. Core 0 writes the shared line 0x020: BusUpgr invalidates the copy in cache 1

Cache 0 finds the line in S, goes to WAIT_GRANT with the command BusUpgr and gets the bus. The snoop FSM of cache 1 sees the BusUpgr, finds the line in S (LOOKUP) and invalidates it (INVALIDATE) two cycles after the command appears. When the transaction ends, cache 0 sets the line to M and performs the write in COMPARE. No data is transferred on the bus.

Result: cache 0 holds the line in M and cache 1 in I. **Passed.**

6.7 Cache-to-Cache Transfer

Sequence: core 1 reads 0x020, which is now modified in cache 0 and invalid in cache 1.

Expected: memory still holds the old value (0x00000020). Cache 0 must supply the new value 0xBBBB0002, and both caches must end in S.

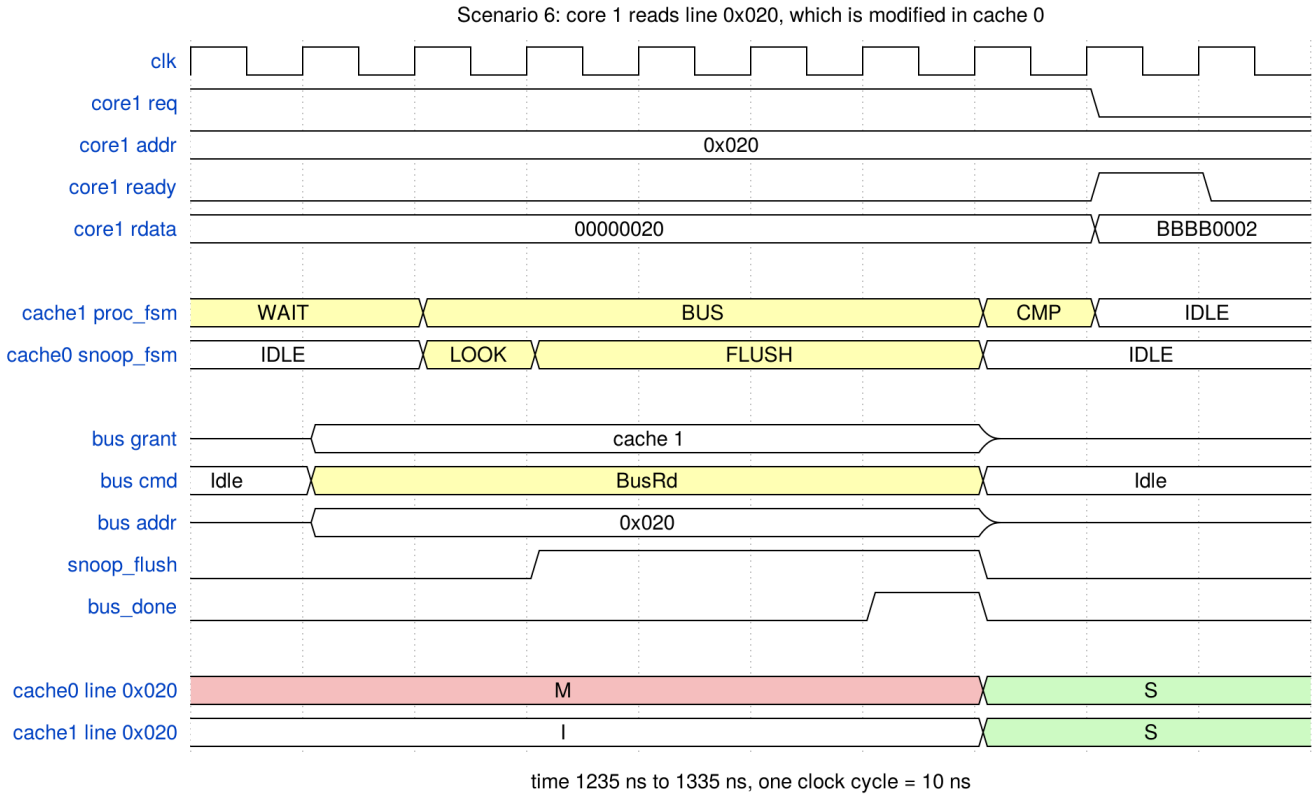


Figure 12. Core 1 reads line 0x020, which is modified in cache 0

Cache 1 misses and issues BusRd. The snoop FSM of cache 0 finds the line in M, goes to FLUSH, asserts `snoop_flush` and drives the line on the bus. When `bus_done` arrives, memory stores the line, cache 1 takes the same data from the bus, and cache 0 moves from M to S.

Result: core 1 reads 0xBBBB0002 (not the old value from memory), and both caches hold the line in S. **Passed.**

6.8 Simultaneous Writes

This scenario is not required by the assignment; it tests the upgrade race described in section 4.3.

Sequence: both cores read 0x030, so both hold the line in S. Then, in the same clock cycle, core 0 writes 0xC0C0C0C0 to 0x030 and core 1 writes 0xC1C1C1C1 to 0x034 (two different words of the same line).

Expected: only one cache can own the line at a time, and neither write may be lost.

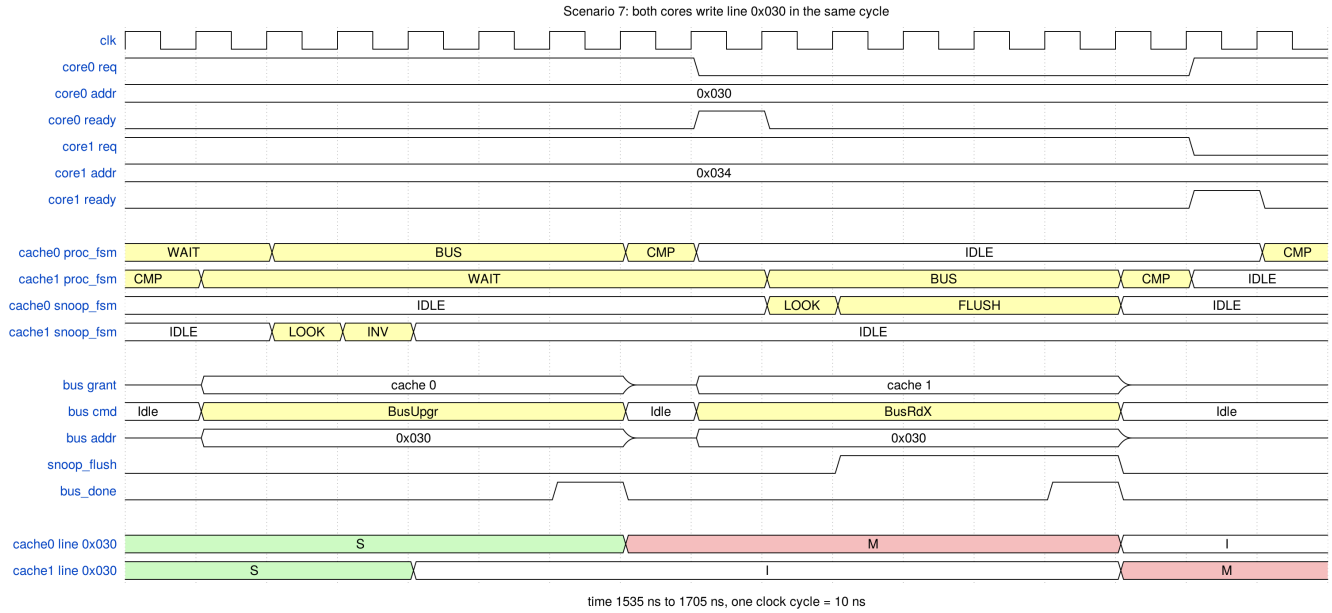


Figure 13. Both cores write line 0x030 in the same cycle

Both caches hit in S and both request the bus with BusUpgr. Cache 1 had the bus last, so the arbiter gives it to cache 0. Cache 0's BusUpgr invalidates the copy in cache 1, and cache 0 moves to M and writes its word. While it waits, cache 1 sees that its line became I and changes its command to BusRdX. When it gets the bus, the snoop FSM of cache 0 flushes the line (which already contains 0xC0C0C0C0) and moves to I. Cache 1 takes the line, moves to M and writes its own word.

Result: after the writes, both cores read 0xC0C0C0C0 at 0x030 and 0xC1C1C1C1 at 0x034, and the line ends in S in both caches. **Passed.**

6.9 Summary of Results

Test	Requirements	Bus transactions	Result
6.2 Hit and miss	R1, R2	BusRd	Passed
6.3 Replacement	R3	BusRd × 2	Passed (LRU and FIFO)

Test	Requirements	Bus transactions	Result
6.4 Write-back	R2, R4	BusRdX, BusRd, Flush, BusRd, BusRd	Passed
6.5 Shared read	R5	BusRd × 2	Passed
6.6 Invalidation	R6	BusUpgr	Passed
6.7 Cache-to-cache transfer	R7	BusRd (data from cache 0)	Passed
6.8 Simultaneous writes	R8	BusUpgr, BusRdX, BusRd	Passed

The access times measured in the waveforms, from the clock edge where the controller accepts the request to the edge where `cpu_ready` rises:

Access	Clock cycles	Explanation
Read hit, or write hit in M	1	COMPARE only
Miss, bus free	9	COMPARE 1 + WAIT_GRANT 2 + BUS_TRANS 5 + COMPARE 1
Write hit in S (BusUpgr)	9	Same path as a miss, without data transfer
Miss with write-back of a modified victim	16	Flush transaction (7 cycles) before the miss
Write that waits for the other core's transaction	15	6 cycles waiting for the bus, then a miss

A miss costs 9 times more than a hit, even with this short memory latency. This shows why a cache helps only when most accesses hit, and why the extra traffic of coherence (BusUpgr, flushes) has a real cost.

7. Conclusions

7.1 Results

The project delivers a working VHDL model of two private, 2-way set-associative, write-back caches kept coherent by the MSI snoopy protocol over a shared, arbitrated bus. All the requirements of the assignment are met and checked automatically by the testbench:

- hits, misses, LRU and FIFO replacements, and write-backs of modified lines;
- the three coherence scenarios: shared read, invalidation on write, and cache-to-cache transfer of a modified line;
- in addition, two cores writing the same shared line in the same cycle, where no write is lost.

The measurements show the cost of each operation: a hit takes 1 cycle, a miss 9 cycles, and a miss that must first write back a modified line 16 cycles. Coherence traffic is not free either: a write to a shared line needs a full bus transaction (BusUpgr) before it can complete.

The simulation log and the C++ viewer make the protocol visible: every step can be followed with its bus transaction, the source of the data and the state change of each line.

7.2 Elements Learned

- **Cache coherence in practice.** Reading about MSI is simple; making it work showed the corner cases, such as a line being invalidated while its owner waits for the bus, or two caches upgrading the same line at the same time.
- **Hardware design in VHDL.** Organizing a design so that only one process writes each signal, replacing a tri-state bus with multiplexers, and making the cache size configurable from one package.
- **Verification.** A self-checking testbench with assertions finds errors much faster than reading waveforms. Logging the simulation to a file made it possible to build a separate tool on top of it.
- **C++ and graphical interfaces.** Building the viewer with Dear ImGui, GLFW and CMake, and organizing it into modules.

7.3 Further Improvements

- **MESI protocol.** An Exclusive state would remove the extra BusUpgr when a core reads and then writes a line that no other cache holds.

- **More cores.** The bus multiplexer and the arbiter could be generalized to N controllers.
- **Split-transaction bus.** The bus is now blocked for the whole memory access (6 cycles); a split bus would let other transactions use it while memory is busy.
- **FPGA implementation.** The design targets simulation. Running it on a board (for example one with an Artix-7 FPGA) would need a hardware request generator in place of the testbench, a top level connected to the board's buttons, switches and LEDs, a constraints file, and a synthesizable memory.
- **Statistics.** Counting hits, misses and bus transactions for longer, random access sequences would allow comparing LRU and FIFO, or different cache sizes, by hit rate.

8. Bibliography

1. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
2. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2014.
3. V. Nagarajan, D. J. Sorin, M. D. Hill and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2020.
4. D. E. Culler, J. P. Singh and A. Gupta, *Parallel Computer Architecture: A Hardware/Software Approach*. San Francisco, CA, USA: Morgan Kaufmann, 1999.
5. A. J. Smith, "Cache memories," *ACM Computing Surveys*, vol. 14, no. 3, pp. 473–530, 1982.
6. J. R. Goodman, "Using cache memory to reduce processor-memory traffic," in *Proc. 10th Annual International Symposium on Computer Architecture (ISCA)*, 1983, pp. 124–131.
7. M. S. Papamarcos and J. H. Patel, "A low-overhead coherence solution for multiprocessors with private cache memories," in *Proc. 11th Annual International Symposium on Computer Architecture (ISCA)*, 1984, pp. 348–354.
8. J. Archibald and J.-L. Baer, "Cache coherence protocols: Evaluation using a multiprocessor simulation model," *ACM Transactions on Computer Systems*, vol. 4, no. 4, pp. 273–298, 1986.
9. P. J. Ashenden, *The Designer's Guide to VHDL*, 3rd ed. Burlington, MA, USA: Morgan Kaufmann, 2008.
10. *IEEE Standard for VHDL Language Reference Manual*, IEEE Std 1076-2019, 2019.

11. Dear ImGui, Bloat-free graphical user interface library for C++. [Online]. Available: https://github.com/ocornut/imgui
12. GLFW, Open-source multi-platform library for OpenGL. [Online]. Available: https://www.glfw.org